Systèmes de Classifieurs Multiples : Théorie, Architecture et Mise en Œuvre de l'Apprentissage d'Ensemble

L'évolution de l'apprentissage automatique a franchi une étape décisive avec l'émergence des systèmes de classifieurs multiples (Multiple Classifier Systems - MCS), plus communément désignés sous le terme d'apprentissage d'ensemble. Cette approche repose sur un paradigme fondamental : la combinaison stratégique de plusieurs modèles de prédiction pour pallier les insuffisances inhérentes à tout classifieur unique.[1] Historiquement, la recherche en intelligence artificielle s'est concentrée sur l'optimisation d'algorithmes individuels. Cependant, l'expérience a démontré qu'un expert unique, aussi sophistiqué soit-il, est souvent limité par ses propres biais structurels et sa sensibilité au bruit des données.[2] L'apprentissage d'ensemble propose de substituer à l'expert solitaire un comité de décideurs dont la sagesse collective surpasse les capacités individuelles.[3]

Ce rapport propose une exploration exhaustive de la combinaison de classifieurs, structurée autour des fondements théoriques, des typologies méthodologiques et des considérations architecturales qui définissent ce domaine de pointe. L'analyse s'attache à démontrer comment la diversité des modèles et la rigueur de leur agrégation permettent d'atteindre des niveaux de performance et de robustesse inaccessibles aux approches classiques.

3.1 Définition et Phases de la Combinaison de Classifieurs

La combinaison de classifieurs est définie comme le processus d'intégration des sorties de plusieurs modèles d'apprentissage afin de générer une décision globale plus fiable et précise.[1] Un système multi-classifieurs est structurellement composé d'un ensemble d'apprenants de base, souvent qualifiés d'"apprenants faibles" (weak learners) lorsqu'ils sont pris isolément, et d'un mécanisme de fusion qui orchestre leurs réponses.[2]

Le cycle de vie d'un tel système se décompose généralement en trois phases critiques identifiées dans la littérature spécialisée [1] :

1. **Génération de l'Ensemble** : Cette phase initiale consiste à produire un pool de modèles de base. L'objectif est de créer des modèles qui ne sont pas seulement performants, mais surtout complémentaires.[1] La complémentarité est assurée par l'introduction de diversité, que ce soit par l'utilisation de différents sous-ensembles de données, de paramètres variés pour un même algorithme, ou de familles d'algorithmes distinctes.[1]
2. **Sélection ou Élagage (Pruning)** : Une fois le pool généré, il peut être nécessaire de réduire la taille de l'ensemble. L'élagage vise à éliminer les modèles redondants ou ceux dont la performance dégrade la décision collective.[1] Cette étape permet non seulement

d'améliorer la précision globale mais aussi de réduire les coûts computationnels lors de l'inférence.[1]

3. **Combinaison ou Fusion** : La phase finale consiste à fusionner les prédictions des modèles sélectionnés pour obtenir la réponse ultime. Le choix de la fonction de combinaison dépend crucialement du type d'information fournie par les classifieurs de base.[1]

Nature des Sorties et Niveaux d'Information

L'efficacité de la combinaison dépend de la richesse des informations transmises par les modèles individuels. On distingue classiquement trois niveaux de sortie [8] :

- **Le niveau Classe (Hard Output)** : Le classifieur fournit uniquement l'étiquette de la classe prédite. C'est l'information la plus élémentaire, où chaque modèle agit comme un votant binaire.[8]
- **Le niveau Rang** : Le classifieur produit une liste ordonnée des classes possibles, de la plus probable à la moins probable. Ce format est particulièrement utile lorsque la classe réelle ne figure pas au premier rang mais reste parmi les premières propositions.[8]
- **Le niveau Mesure (Soft Output)** : C'est le niveau le plus riche. Le classifieur associe à chaque classe une valeur numérique, telle qu'une probabilité ou un score de confiance. Cette mesure quantifie l'incertitude du modèle et permet des fusions pondérées beaucoup plus fines.[8]

Le tableau suivant présente une comparaison structurelle des types d'informations en sortie :

Type de Sortie	Information Contenue	Complexité de Fusion	Cas d'Utilisation Typique
Classe	Étiquette discrète uniquement	Faible	Vote majoritaire simple
Rang	Préférences relatives ordonnées	Moyenne	Méthode de Borda, systèmes de recommandation
Mesure	Probabilités, scores de confiance	Élevée	Moyenne pondérée, Stacking, Bayes

3.2 Motivations Théoriques et Justifications Statistiques

La supériorité des systèmes multi-classifieurs n'est pas une simple observation empirique ; elle s'appuie sur des piliers théoriques robustes, notamment le Théorème du Jury de Condorcet et la décomposition biais-variance.[5]

Le Théorème du Jury de Condorcet

Formulé à l'origine dans le contexte des sciences sociales, ce théorème constitue une base théorique fondamentale pour l'apprentissage d'ensemble.[14] Il postule que si un groupe

d'individus doit prendre une décision binaire (par exemple, classer un objet comme A ou B) et que chaque individu a une probabilité p supérieure à $0,5$ de choisir la bonne réponse, alors la probabilité que la décision de la majorité soit correcte augmente avec le nombre d'individus.[14]
Mathématiquement, si L est le nombre de classifieurs indépendants et p leur précision individuelle, la probabilité P que l'ensemble prenne la bonne décision par vote majoritaire tend vers 1 lorsque L tend vers l'infini.[14] À l'inverse, si $p < 0,5$, l'ajout de classifieurs réduit la performance globale. Cette démonstration justifie l'utilisation d'"apprenants faibles" pour construire des systèmes puissants, à condition qu'ils soient plus performants qu'un tirage aléatoire et qu'ils soient suffisamment indépendants.[9]

Décomposition Biais-Variance

D'un point de vue statistique, l'erreur de généralisation d'un modèle d'apprentissage peut être décomposée en trois composantes : le biais au carré, la variance et l'erreur irréductible (bruit).[16]
La formule de l'erreur quadratique moyenne (MSE) est :

$$E[(y - \hat{f}(x))^2] = Bias[\hat{f}(x)]^2 + Var[\hat{f}(x)] + \sigma^2$$
Où σ^2 représente le bruit inhérent aux données.[5]

- **Le Biais** mesure l'écart entre la prédiction moyenne du modèle et la valeur réelle. Un biais élevé est le signe d'un modèle trop simpliste qui ne capture pas la complexité des données (underfitting).[16]
- **La Variance** mesure la dispersion des prédictions pour différents jeux de données d'apprentissage. Une variance élevée indique que le modèle est trop sensible aux fluctuations et au bruit des données (overfitting).[16]

L'apprentissage d'ensemble permet de naviguer dans ce compromis (Bias-Variance Tradeoff). Les méthodes de type Bagging agissent principalement en réduisant la variance par la moyenne de modèles complexes, tandis que les méthodes de Boosting se concentrent sur la réduction du biais en agrégeant itérativement des modèles simples pour corriger leurs erreurs systématiques.[13]

Les Problèmes de Dietterich

Thomas Dietterich a identifié trois raisons fondamentales expliquant pourquoi un ensemble peut surpasser un modèle unique [2] :

1. **Le Problème Statistique** : Lorsque le volume de données est faible par rapport à la taille de l'espace des hypothèses, l'algorithme peut trouver plusieurs modèles qui s'ajustent parfaitement aux données d'entraînement. En les combinant, l'ensemble réduit le risque de choisir par erreur un modèle qui généralise mal.[2]
2. **Le Problème Computationnel** : De nombreux algorithmes (comme les réseaux de neurones) utilisent des heuristiques de recherche locale qui peuvent rester bloquées dans des optima locaux. Un ensemble effectuant plusieurs recherches à partir de points de départ différents a plus de chances de trouver une meilleure approximation globale.[2]

3. **Le Problème de Représentation** : Parfois, la véritable fonction cible ne peut être représentée par aucun des modèles de l'espace d'hypothèses choisi. La combinaison de plusieurs modèles permet d'étendre cet espace et d'approcher des fonctions plus complexes.[2]

3.3 Typologies des Méthodes d'Ensemble : Approfondissement

Les méthodes d'ensemble se divisent principalement en approches parallèles (Bagging), séquentielles (Boosting) et hiérarchiques (Stacking).[6]

3.3.1 Bagging (Bootstrap Aggregating)

Le Bagging vise à réduire la variance en entraînant plusieurs modèles indépendants en parallèle sur des échantillons de données légèrement différents.[18]

Fonctionnement détaillé :

1. **Bootstrapping** : On crée L nouveaux jeux de données par tirage aléatoire avec remise. Environ $63,2\%$ des données originales apparaissent dans un échantillon donné, le reste constituant les données **Out-of-Bag (OOB)**.[24]
2. **Entraînement parallèle** : L classifieurs (généralement des arbres de décision profonds) sont entraînés indépendamment sur chaque échantillon.[6]
3. **Agrégation** : Les résultats sont fusionnés par vote majoritaire (classification) ou moyenne simple (régression).[23]

Exemple : La Forêt Aléatoire (Random Forest) C'est l'extension la plus célèbre du Bagging. En plus du bootstrapping, elle introduit une randomisation des caractéristiques (features) : à chaque division d'un nœud de l'arbre, seul un sous-ensemble aléatoire de variables est considéré. Cela décorrèle les arbres et maximise la réduction de variance.[1]

Schéma Pédagogique du Bagging : Imaginez un flux horizontal. À gauche, le "Dataset Original". Il se divise en plusieurs flèches parallèles vers des "Bags" (échantillons bootstrap). Chaque Bag alimente un "Modèle de Base" indépendant. Toutes les sorties des modèles convergent enfin vers un bloc central "Agrégation (Vote/Moyenne)" qui produit la "Prédiction Finale".[6]

3.3.2 Boosting

Le Boosting est une approche séquentielle où chaque modèle tente de corriger les erreurs de son prédécesseur, réduisant ainsi principalement le biais.[18]

Mécanisme d'AdaBoost (Adaptive Boosting) :

1. **Initialisation** : On attribue un poids identique $w_i = 1/n$ à chaque exemple.[3]
2. **Itération** : On entraîne un modèle faible (souvent un "Stump", arbre de profondeur 1). On calcule son erreur pondérée ϵ.[15]
3. **Mise à jour des poids** : Les exemples mal classés voient leur poids augmenter, forçant le modèle suivant à se concentrer sur eux. Les exemples bien classés voient leur poids

diminuer.[20]

4. **Combinaison** : La décision finale est une somme pondérée des modèles par leur coefficient de confiance α_t.[20]

Gradient Boosting (XGBoost, LightGBM) : Plutôt que de repondérer les points, on entraîne chaque nouveau modèle pour prédire le **résidu** (l'erreur) du modèle précédent. On descend ainsi le gradient de la fonction de perte pour minimiser l'erreur globale étape par étape.[4]

Schéma Pédagogique du Boosting : Représentez une chaîne série. "Modèle 1" analyse les données. Les erreurs sont transmises à un bloc "Mise à jour des Poids/Calcul des Résidus", qui modifie l'importance des données pour le "Modèle 2", et ainsi de suite. À la fin, tous les modèles contribuent avec un poids différent à la "Décision Finale".[18]

3.3.3 Stacking (Stacked Generalization)

Le Stacking utilise un modèle d'apprentissage pour apprendre comment combiner au mieux les prédictions de plusieurs modèles de base.[1]

Architecture en couches :

- **Niveau 0 (Base-Learners)** : Des modèles hétérogènes (ex: SVM, Forêt Aléatoire, KNN) sont entraînés sur les données.[1]
- **Méta-caractéristiques** : Les prédictions de ces modèles servent de nouvelles variables d'entrée.[4]
- **Niveau 1 (Méta-modèle)** : Un dernier classifieur (souvent une régression logistique) apprend à pondérer ces prédictions.[26]

Le rôle de la validation croisée (StackingCV) : Pour éviter le surapprentissage (le méta-modèle pourrait favoriser les modèles qui ont "mémorisé" le train), on utilise des prédictions **Out-of-Fold**. Chaque modèle de base prédit des données qu'il n'a pas vues durant son entraînement partiel, garantissant que le méta-modèle apprend sur des performances réelles de généralisation.[15]

Schéma Pédagogique du Stacking : Imaginez une structure pyramidale. La base est le "Dataset original". Il alimente plusieurs "Modèles de base" (Niveau 0) différents. Leurs flèches de sortie pointent vers un "Nouveau Dataset de prédictions". Ce dernier est l'unique entrée du "Méta-modèle" (Niveau 1) situé au sommet, qui délivre la "Prédiction Finale".[13]

3.3.4 Voting (Vote Majoritaire vs Vote Probabiliste)

Le vote est la méthode de fusion la plus simple pour les modèles parallèles.

1. Hard Voting (Vote Majoritaire) :

Chaque modèle donne une étiquette de classe. On choisit la classe qui a reçu le plus de suffrages.

- *Exemple* : Modèle A -> Chien, Modèle B -> Chat, Modèle C -> Chien. Résultat final = Chien.

2. Soft Voting (Vote Probabiliste) :

On calcule la moyenne des probabilités prédites pour chaque classe. Cette méthode est plus précise car elle accorde plus d'importance aux modèles "sûrs d'eux".

Exemple Numérique Comparatif :

Considérons deux classes (Chat, Chien) et trois modèles :

- Modèle 1 : Probabilité [Chien=0.4, Chat=0.6] -> Prédit **Chat**
- Modèle 2 : Probabilité [Chien=0.4, Chat=0.6] -> Prédit **Chat**
- Modèle 3 : Probabilité [Chien=0.9, Chat=0.1] -> Prédit **Chien**
- **En Hard Voting** : Chat (2 voix) bat Chien (1 voix). Résultat = **Chat**.
- **En Soft Voting** : Moyenne Chien = $(0.4+0.4+0.9)/3 = \mathbf{0.56}$; Moyenne Chat = $(0.6+0.6+0.1)/3 = \mathbf{0.43}$. Résultat = **Chien**.
  Le Soft Voting a permis ici à la forte conviction du Modèle 3 de l'emporter sur l'incertitude des deux autres.

3.5 Aspects Architecturaux des MCS

L'architecture d'un système multi-classifieurs définit la manière dont les informations circulent entre les composants et la façon dont les ressources sont exploitées.[8]

Architectures de Contrôle de Flux

La littérature distingue trois organisations principales [8] :

1. **Architecture Parallèle** : Tous les classifieurs opèrent indépendamment et simultanément. La fusion intervient à la fin. Cette structure est neutre (aucun classifieur n'influence l'autre) et hautement parallélisable sur du matériel multi-cœurs.[7]
2. **Architecture Séquentielle (Série)** : Les classifieurs sont organisés en cascade. Un premier classifieur rapide effectue une prédiction ; s'il échoue ou exprime une faible confiance, l'exemple est passé à un second modèle plus complexe.[8] Cela permet d'économiser des ressources de calcul en ne sollicitant les modèles coûteux que pour les cas difficiles.[10]
3. **Architecture Hybride** : Elle combine les deux approches. Par exemple, un premier étage parallèle peut générer un consensus, dont le résultat est validé par un classifieur de contrôle séquentiel.[8]

Systèmes de Sélection Dynamique (DS)

Une tendance majeure consiste à ne pas utiliser tous les classifieurs pour chaque exemple, mais à sélectionner "à la volée" le ou les modèles les plus compétents.[1]

- **Logic de Compétence Locale** : On définit une "région de compétence" autour d'un exemple test (en utilisant ses voisins les plus proches dans le jeu d'entraînement). On évalue ensuite quels modèles du pool ont été les plus performants dans cette zone spécifique.[33]
- **DCS (Dynamic Classifier Selection)** : On choisit l'unique meilleur modèle pour l'exemple donné.[24]
- **DES (Dynamic Ensemble Selection)** : On construit un sous-ensemble de modèles experts pour cette région particulière.[33]

3.4 Analyse Comparative des Approches

Le choix d'une méthode d'ensemble dépend des contraintes du projet (volume de données, temps de réponse, précision requise).[6]

Critère	Bagging	Boosting	Stacking
Objectif Principal	Réduire la variance	Réduire le biais	Maximiser la précision
Type de Modèles	Homogènes (Arbres profonds)	Homogènes (Arbres peu profonds)	Hétérogènes (Divers types)
Entraînement	Parallèle (Indépendant)	Séquentiel (Dépendant)	Parallèle + Méta-apprentissage
Sensibilité au Bruit	Faible (Robuste)	Élevée (Risque de sur-ajuster)	Variable
Risque d'Overfitting	Faible	Élevé (si non régularisé)	Moyen (si données limitées)
Coût de calcul	Efficace (parallélisable)	Lent (entraînement en série)	Très élevé

3.5 Avantages et Limites de la Combinaison

Avantages

1. **Précision Supérieure** : Les ensembles dépassent quasi systématiquement les modèles uniques en combinant des perspectives complémentaires.[6]
2. **Généralisation et Robustesse** : En lissant les prédictions, les systèmes MCS sont moins sensibles aux anomalies statistiques d'un seul jeu de données.[5]
3. **Traitement des Données Complexes** : Ils permettent de diviser un problème complexe en sous-tâches traitées par des experts dédiés (Divide and Conquer).[3]

Limites

1. **Coût en Ressources** : L'entraînement de centaines de modèles consomme du temps CPU, de la mémoire et de l'énergie.
2. **Opaque ("Black Box")** : Il est extrêmement difficile d'expliquer pourquoi un ensemble de 500 arbres de décision a pris une décision spécifique.[2]
3. **Complexité de Maintenance** : Gérer un pipeline avec plusieurs modèles hétérogènes demande une infrastructure plus lourde qu'un modèle monolithique.[6]

En conclusion, la combinaison de classifieurs représente une approche mature et extrêmement puissante de l'apprentissage automatique. En orchestrant judicieusement la diversité et en s'appuyant sur des fondements statistiques rigoureux, les systèmes multi-classifieurs permettent de transformer une collection de modèles imparfaits en un outil de décision d'une fiabilité exceptionnelle.[1]

Sources des citations

1. Multiple Classifier Systems — a brief introduction | luisfredgs - Medium, consulté

le février 23, 2026, https://medium.com/luisfredgs/multiple-classifier-systems-a-brief-introduction-71238d9c794f
2. Ensemble Classifier | Data Mining - GeeksforGeeks, consulté le février 23, 2026, https://www.geeksforgeeks.org/machine-learning/ensemble-classifier-data-mining/
3. Ensemble Learning: Key Techniques Explained - Turing College, consulté le février 23, 2026, https://www.turingcollege.com/blog/ensemble-learning-2
4. Bagging vs Boosting vs Stacking: Which Ensemble Method Wins in ..., consulté le février 23, 2026, https://machinelearningmastery.com/bagging-vs-boosting-vs-stacking-which-ensemble-method-wins-in-2025/
5. Bagging, Boosting and Stacking: Ensemble Learning in ML Models | by Kaleemullah Younas | Medium, consulté le février 23, 2026, https://medium.com/@kaleemullahyounas123/bagging-boosting-and-stacking-ensemble-learning-in-ml-models-8eb2f73afb07
6. Bagging, Boosting and Stacking: Ensemble Learning in ML Models - Analytics Vidhya, consulté le février 23, 2026, https://www.analyticsvidhya.com/blog/2023/01/ensemble-learning-methods-bagging-boosting-and-stacking/
7. Exploration de l'apprentissage d'ensemble : Son rôle dans l'IA et la ML | Ultralytics, consulté le février 23, 2026, https://www.ultralytics.com/fr/blog/exploring-ensemble-learning-and-its-role-in-ai-and-ml
8. Cours CombinaisonClassifieurs | PDF - Scribd, consulté le février 23, 2026, https://fr.scribd.com/document/811893910/Cours-CombinaisonClassifieurs
9. Ensemble learning - Wikipedia, consulté le février 23, 2026, https://en.wikipedia.org/wiki/Ensemble_learning
10. Combinaison de Classifieurs - Laurent HEUTTE - Free, consulté le février 23, 2026, http://lheutte.free.fr/rdf/ArchiCombi.pdf
11. Dynamic Ensemble Selection and Instantaneous Pruning for Regression, consulté le février 23, 2026, https://www.esann.org/sites/default/files/proceedings/legacy/es2014-173.pdf
12. From dynamic classifier selection to dynamic ensemble selection - biz.uiowa.edu, consulté le février 23, 2026, https://www.biz.uiowa.edu/faculty/nstreet/ko07.pdf
13. On Theory Behind Ensemble Learning Techniques | by Pavan B Govindaraju - Medium, consulté le février 23, 2026, https://gpavanb.medium.com/on-theory-behind-ensemble-learning-techniques-156eb63acdaa
14. Théorème du jury - Wikipédia, consulté le février 23, 2026, https://fr.wikipedia.org/wiki/Th%C3%A9or%C3%A8me_du_jury
15. Maîtriser l'Ensemble Learning en Machine Learning : Techniques et Applications - YouTube, consulté le février 23, 2026, https://www.youtube.com/watch?v=bx_tZ9t7Ehs
16. Bias–variance tradeoff - Wikipedia, consulté le février 23, 2026,

https://en.wikipedia.org/wiki/Bias%E2%80%93variance_tradeoff
17. Bias/Variance Tradeoff and Ensemble Methods - The University of Texas at Dallas, consulté le février 23, 2026, https://personal.utdallas.edu/~vibhav.gogate/ml/2020f/lectures/EnsembleMethods.pdf
18. Understanding the Ensemble method Bagging and Boosting - Great Learning, consulté le février 23, 2026, https://www.mygreatlearning.com/blog/bagging-boosting/
19. 'Bias–Variance Tradeoff' and 'Ensemble Methods' Explained : r/learnmachinelearning, consulté le février 23, 2026, https://www.reddit.com/r/learnmachinelearning/comments/1q32ugd/biasvariance_tradeoff_and_ensemble_methods/
20. Machine Learning TD - Méthodes Ensemblistes - Laboratoire ERIC, consulté le février 23, 2026, https://eric.univ-lyon2.fr/gmetzler/courses/contents/machinelearning/td4.pdf
21. What is ensemble learning (bagging vs boosting vs stacking)? | by NS Academy - Medium, consulté le février 23, 2026, https://medium.com/@sharetonschool/what-is-ensemble-learning-bagging-vs-boosting-vs-stacking-c3c8cc94a3d5
22. Bagging, boosting and stacking in machine learning - Stats StackExchange, consulté le février 23, 2026, https://stats.stackexchange.com/questions/18891/bagging-boosting-and-stacking-in-machine-learning
23. What is ensemble learning? - IBM, consulté le février 23, 2026, https://www.ibm.com/think/topics/ensemble-learning
24. DESlib: A Dynamic ensemble selection library in Python - Journal of Machine Learning Research, consulté le février 23, 2026, https://jmlr2020.csail.mit.edu/papers/volume21/18-144/18-144.pdf
25. A Unified Theory of Diversity in Ensemble Learning | Request PDF - ResearchGate, consulté le février 23, 2026, https://www.researchgate.net/publication/367019941_A_Unified_Theory_of_Diversity_in_Ensemble_Learning
26. What Is Bagging? | IBM, consulté le février 23, 2026, https://www.ibm.com/think/topics/bagging
27. scikit-learn-contrib/DESlib: A Python library for dynamic classifier and ensemble selection, consulté le février 23, 2026, https://github.com/scikit-learn-contrib/DESlib
28. [Théorie] Apprentissages d'ensemble : bagging, boosting et stacking - Medium, consulté le février 23, 2026, https://medium.com/@henryhenry_3807/th%C3%A9orie-apprentissages-densemble-bagging-boosting-et-stacking-f3c8d2273edf
29. Learn Voting: Hard and Soft | Section - Codefinity, consulté le février 23, 2026, https://codefinity.com/courses/v2/3172b303-2b09-492b-8bb9-5bbd1bdcef09/81f3a01b-d537-4478-bc23-3ea1895e4f1f/d66aa7bd-877e-4808-ad40-aa051eabbc4a

30. Ensemble Learning - GeeksforGeeks, consulté le février 23, 2026, https://www.geeksforgeeks.org/machine-learning/a-comprehensive-guide-to-ensemble-learning/
31. Ensemble Methods for Machine Learning, consulté le février 23, 2026, https://www.cs.cmu.edu/~wcohen/10-601/ensembles.pdf
32. Voting Classifier - GeeksforGeeks, consulté le février 23, 2026, https://www.geeksforgeeks.org/machine-learning/voting-classifier/
33. Welcome to DESlib documentation! — deslib 0.4.dev documentation, consulté le février 23, 2026, https://deslib.readthedocs.io/
34. (PDF) Comparative analysis of algorithmic approaches in ensemble learning: bagging vs. boosting - ResearchGate, consulté le février 23, 2026, https://www.researchgate.net/publication/396095365_Comparative_analysis_of_algorithmic_approaches_in_ensemble_learning_bagging_vs_boosting
35. What is Ensemble Learning? | Bagging, Boosting, Stacking | SabrePC Blog, consulté le février 23, 2026, https://www.sabrepc.com/blog/deep-learning-and-ai/what-is-ensemble-learning-bagging-boosting-stacking
36. Bagging, Boosting, and Stacking in Machine Learning | Baeldung on Computer Science, consulté le février 23, 2026, https://www.baeldung.com/cs/bagging-boosting-stacking-ml-ensemble-models